

# Semi-Secret Wars

## AI Development Guide (ADG)

**Version:** 0.1

**Status:** Living Document

---

### 1. Purpose

This document defines the development workflow for **Semi-Secret Wars**.

It exists to guide AI-assisted development using **Table 5** together with the latest stable version of **Godot**.

The objective is to build the game through small, testable milestones while maintaining high code quality, low AI costs, and a scalable project architecture.

This document defines **how the game should be developed**, not how the game works.

---

### 2. Project Goals

Development should always prioritize:

- Playable builds
  - Small milestones
  - Modular systems
  - Maintainable code
  - Low AI token usage
  - Easy testing
  - Easy future expansion
- 

### 3. Roles

#### Designer

Richard

Responsibilities:

- Game Design
  - Gameplay decisions
  - Feature approval
  - QA testing
  - Production direction
- 

## **Fable 5**

Responsibilities:

- Technical implementation
- Architecture recommendations
- Godot best practices
- Performance considerations
- Documentation updates

Fable may recommend improvements, but creative decisions always belong to the designer.

---

## **4. Development Philosophy**

Always build the smallest possible feature that moves the project forward.

Never implement multiple major systems in a single milestone.

Every milestone should result in a playable project.

---

## **5. Development Workflow**

For every development session:

1. Read this document.
  2. Review the current Game Design Document.
  3. Review the current milestone.
  4. Explain the implementation plan.
  5. Implement only the approved milestone.
  6. Verify the project compiles correctly.
  7. Explain how to test the implementation.
  8. Wait for designer approval before continuing.
-

## 6. Milestone Rules

Each milestone should solve only one problem.

Examples:

- Create Main Menu
- Create Hero Scene
- Add Hero Movement
- Add Minion AI
- Add Health System

Avoid combining unrelated systems.

---

## 7. Vertical Slice First

Development should prioritize a playable prototype before expanding content.

General order:

- Project setup
  - Battlefield
  - Hero
  - Enemy
  - Combat
  - Objectives
  - Rewards
  - Progression
  - Expansion
- 

## 8. Godot Best Practices

Use the latest stable Godot version.

Prefer:

- Typed GDScript
- Composition over inheritance
- Signals
- Resources for configurable data
- Reusable scenes
- Modular architecture

Avoid:

- Hardcoded gameplay values
  - Large monolithic scripts
  - Duplicate code
  - Tight coupling
  - Unnecessary singletons
- 

## 9. Placeholder Policy

Until final production assets exist:

- Use Godot primitives whenever possible.
- Use placeholder UI.
- Use placeholder characters.
- Use placeholder effects.
- Do not depend on external assets.

All placeholders should be easy to replace.

---

## 10. Animation Guidelines

Animations should intentionally be simple.

Visual inspiration:

- South Park
- Paper puppets
- Child notebook drawings

Preferred techniques:

- Slight vertical bobbing
- Simple rotation
- Basic scaling
- Minimal keyframes

Complex animation systems are unnecessary for the prototype.

---

## 11. Coding Standards

Scripts should:

- Have a single responsibility.
- Be modular.
- Be reusable.
- Be readable.
- Be documented when necessary.

Gameplay values should be configurable rather than hardcoded.

---

## 12. Documentation

After every approved milestone, update documentation if applicable.

Possible documentation includes:

- Architecture
- Balance
- Localization
- Backlog
- Changelog

Only update documents affected by the milestone.

---

## 13. Token Optimization

AI token usage is considered a project resource.

Fable should:

- Reuse existing code whenever possible.
- Modify existing systems instead of rewriting them.
- Avoid unnecessary explanations.
- Generate only code required for the current milestone.
- Avoid duplicate implementations.

Always prefer extending existing systems over replacing them.

---

## 14. Token Usage Report

At the end of every development session, provide a short report including:

- Milestone completed
  - Files created
  - Files modified
  - Documentation updated
  - Estimated implementation complexity
  - Estimated AI context size
  - Suggestions for reducing future token usage
- 

## 15. Testing Workflow

After implementing a milestone:

- Verify the project runs.
- Explain what should be tested.
- Wait for designer feedback.

Do not continue until approval is received.

---

## 16. Development Backlog

Maintain a prioritized backlog.

Suggested fields:

- Feature
  - Priority
  - Status
  - Dependencies
  - Notes
- 

## 17. Living Documentation

Maintain documentation throughout development.

Important documentation includes:

- Balance
- Localization
- Architecture
- Production Notes
- Changelog

Documentation should always reflect the current state of the project.

---

## 18. Future Team Readiness

Organize the project so that future developers can contribute without major restructuring.

Favor clarity over clever implementations.

---

## 19. Unknown Information

If required information is unavailable:

- Do not invent it.
  - Mark it as **TBD**.
  - Ask the designer before implementing assumptions.
- 

## 20. Success Criteria

A milestone is considered complete when:

- The project compiles.
  - The feature works.
  - The feature is testable.
  - Documentation is updated if necessary.
  - The designer approves the implementation.
-

## Revision History

| Version | Date            | Notes              |
|---------|-----------------|--------------------|
| 0.1     | Initial version | Project foundation |